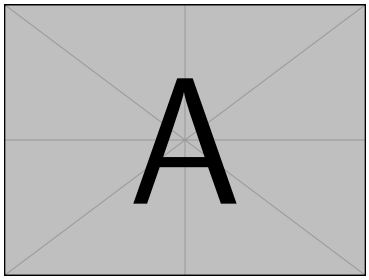
HighPerformance

Performance Management & Bonus Calculation System



Project Documentation
Version 1.0 — February 2026

Development Team
Integration Architecture Project

Enterprise-Grade System

March 5, 2026

Abstract

HighPerformance is a comprehensive enterprise-grade system designed for sales organizations to automate bonus calculations, manage performance metrics, and provide real-time dashboards with multi-level approval workflows. This document provides complete technical documentation including architecture, API reference, database schemas, integration guides, and deployment procedures.

Property	**Value**
Project Type	Web Application
Technology Stack	Node.js, Angular, MongoDB, Camunda
Status	Active Development
Last Updated	February 11, 2026

Contents

Abstract **1**

1 Executive Summary **4**
1.1 Project Overview 4
1.2 Technology Stack 4
1.3 Key Stakeholders 4

2 System Architecture **5**
2.1 High-Level Architecture 5
2.2 Component Architecture 6
2.3 Data Flow Diagram 6

3 Project Structure **7**
3.1 Directory Hierarchy 7

4 Database Models **8**
4.1 Data Model Overview 8
4.2 User Model 9
4.3 Bonus Computation Model 9

5 API Reference **10**
5.1 Authentication 10
5.1.1 Login Endpoint 10
5.2 Salesmen Management 10
5.2.1 List Salesmen 10
5.2.2 Create Salesman 10
5.3 Bonus Computation 11
5.3.1 Compute Bonus 11
5.3.2 Approval Endpoints 11

6 Workflow Orchestration **12**
6.1 Bonus Approval Workflow 12
6.2 Camunda Integration 12

7 Setup & Deployment **13**
7.1 Prerequisites 13
7.2 Development Setup 13
7.2.1 Step 1: Infrastructure 13
7.2.2 Step 2: Backend Setup 13
7.2.3 Step 3: Frontend Setup 13
7.3 Production Deployment 14

7.3.1 Docker Build . . . 14
7.3.2 Environment Configuration . . . 14

8 External Integrations **15**
8.1 OpenCRX Integration . . . 15

9 Testing & Quality **16**
9.1 Test Architecture . . . 16
9.2 Running Tests . . . 16
9.3 Test Suites . . . 16

10 Troubleshooting & Support **17**
10.1 Common Issues . . . 17
10.1.1 MongoDB Connection Failed . . . 17
10.1.2 Backend Won't Start . . . 17
10.1.3 Frontend Build Errors . . . 17
10.2 Performance Optimization . . . 18
10.3 Security Checklist . . . 18

11 Bonus Calculation Algorithm **19**
11.1 Overview . . . 19
11.2 Social Performance Calculation . . . 19
11.2.1 Formula . . . 19
11.2.2 Example . . . 19
11.2.3 Implementation . . . 20
11.3 Order-Based Bonus Calculation . . . 20
11.3.1 Formula . . . 20
11.3.2 Example . . . 20
11.4 Total Bonus Computation . . . 20
11.5 Bonus Constraints . . . 20

12 Workflow & Approval Process **21**
12.1 BPMN Workflow Overview . . . 21
12.1.1 Workflow State Machine . . . 21
12.2 Workflow States . . . 22
12.3 Approval Task Details . . . 22
12.3.1 CEO Review Task . . . 22
12.3.2 HR Review Task . . . 22
12.4 Workflow API Endpoints . . . 23
12.5 Integration with Services . . . 23
12.5.1 Camunda Service Integration . . . 23
12.6 Notifications & Escalations . . . 23
12.6.1 Email Notifications . . . 23
12.6.2 Escalation Rules . . . 23
12.7 Workflow Monitoring . . . 24

A Quick Reference **25**
A.1 API Endpoints Summary . . . 25
A.2 Demo Credentials . . . 25
A.3 Useful Commands . . . 26

B Glossary **27**

Chapter 1 Executive Summary

1.1 Project Overview

HighPerformance addresses enterprise requirements for performance management and bonus automation. The system provides:

- **Automated Bonus Calculation** — Multi-component computations based on performance metrics
- **Multi-Level Approval Workflow** — CEO and HR approval with process orchestration
- **Real-Time Dashboards** — Role-based analytics and visualization
- **Enterprise Integration** — Connectors for OpenCRX and Camunda BPM
- **Security-First Design** — JWT authentication, RBAC, encrypted communications
- **Audit Compliance** — Complete logging and historical tracking

1.2 Technology Stack

Layer	Technology	Version
Frontend	Angular	16+
Backend	Express.js (Node.js)	4.x
Database	MongoDB	6.x+
Workflow Engine	Camunda BPM	Latest
Authentication	JWT	jsonwebtoken 9.0+
API Documentation	OpenAPI	3.0
Containerization	Docker	Latest

Table 1.1: Technology Stack Components

1.3 Key Stakeholders

Supported Roles

CEO Compute bonuses, approve CEO-level, access executive dashboards

HR Manage salesmen, create performance records, approve HR-level

Salesman View personal bonuses (when released), submit qualifications

Admin System configuration, user management, integrations

Chapter 2 System Architecture

2.1 High-Level Architecture

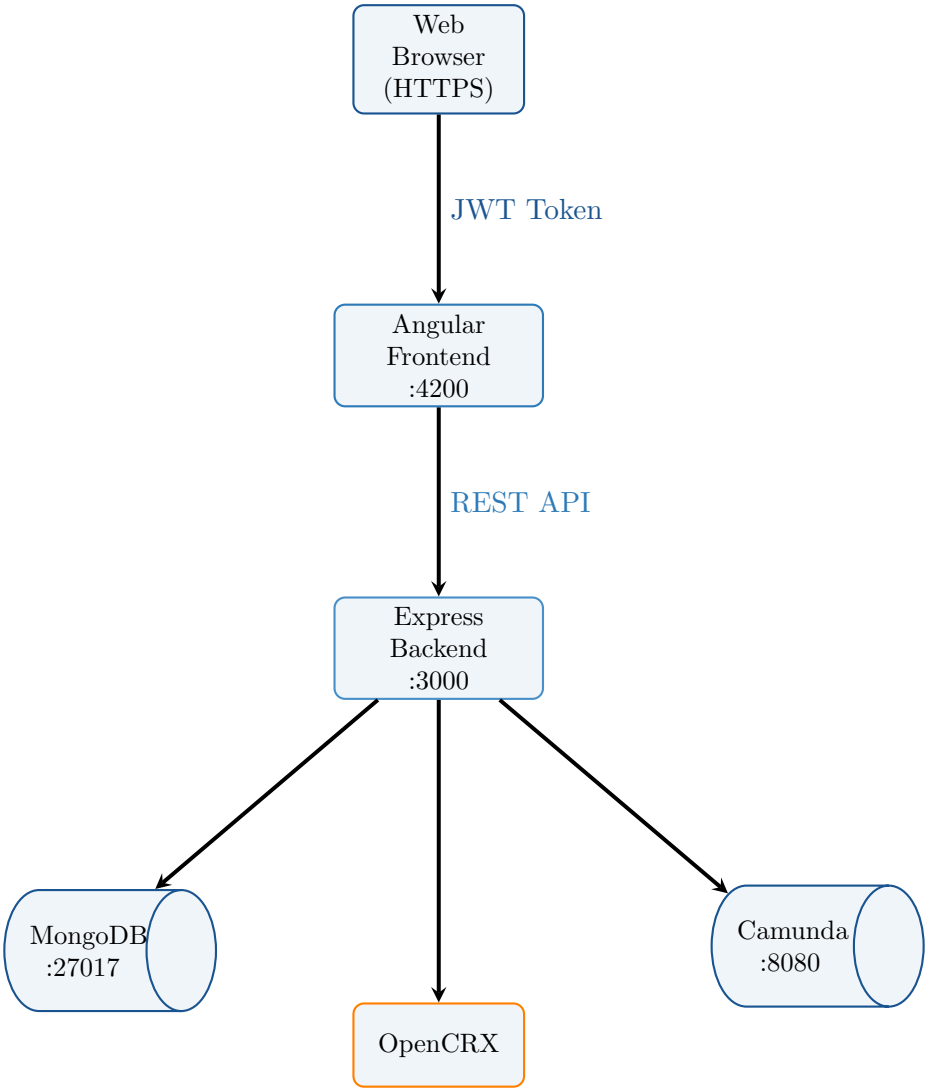


Figure 2.1: System Architecture Overview

2.2 Component Architecture

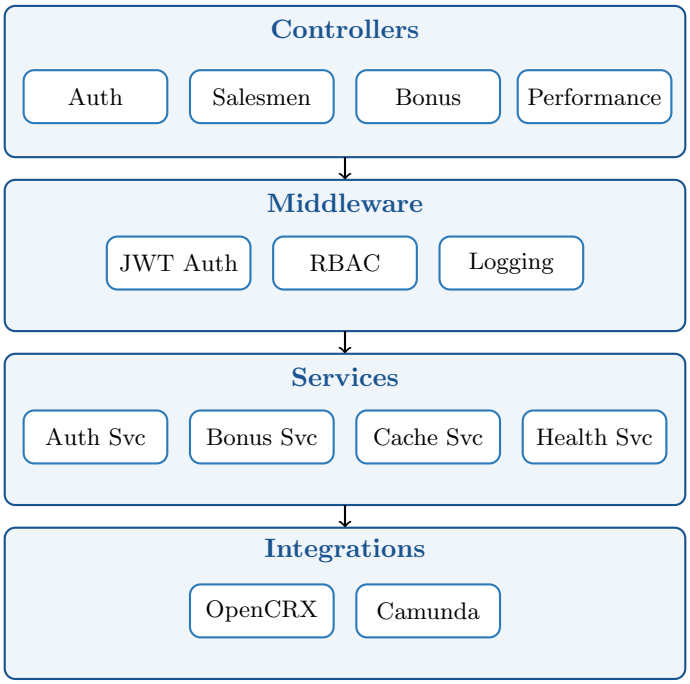


Figure 2.2: Express Backend Layer Architecture

2.3 Data Flow Diagram

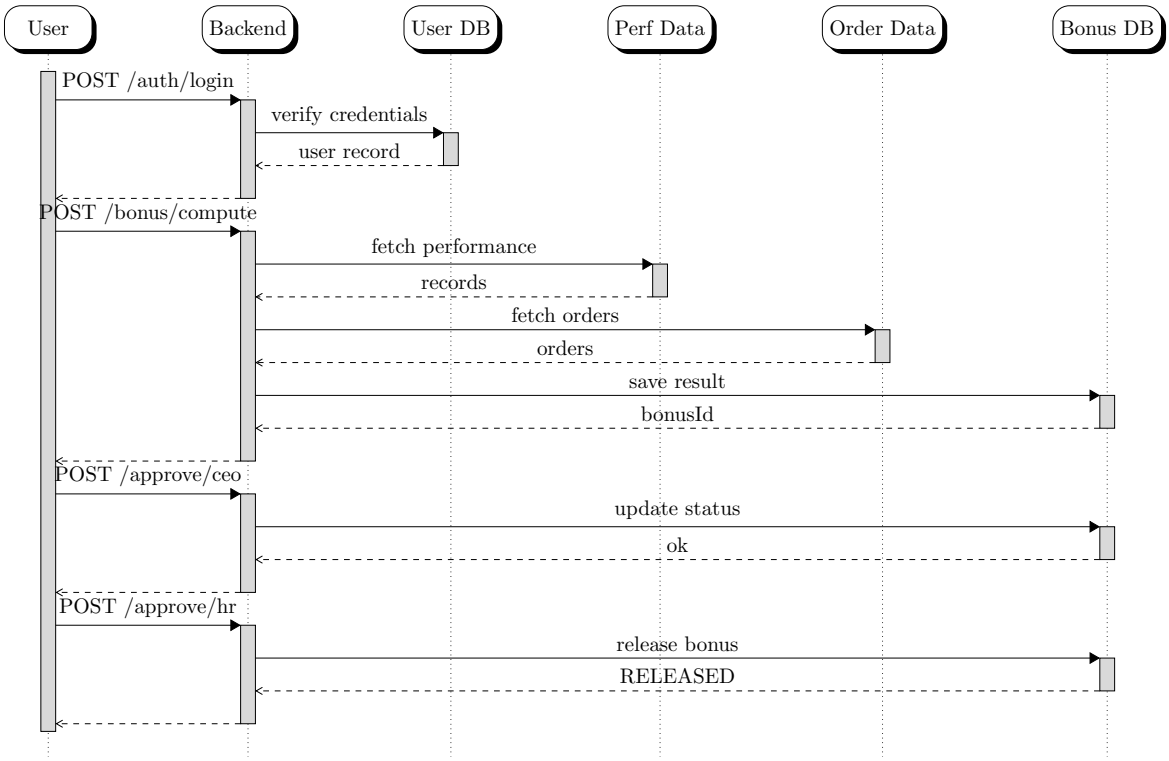


Figure 2.3: Bonus Computation Data Flow

Chapter 3 Project Structure

3.1 Directory Hierarchy

```
MaiPhucSan-InAr-develop/
  backend/                    # Node.js Express API
    src/
      controllers/            # HTTP request handlers
      middleware/             # Express middleware
      models/                 # Mongoose schemas
      routes/                 # API routes
      services/               # Business logic
      db/                     # Database config
      config/                 # Configuration
      app.js
    test/                     # Test files
    package.json
  frontend/                   # Angular application
    src/
      app/                    # Components & services
      environments/           # Config files
      main.ts
    angular.json
  docs/                       # Documentation
  workflow/                   # BPMN processes
  docker-compose.yml          # Infrastructure
```

Listing 3.1: Project Directory Layout

Chapter 4 Database Models

4.1 Data Model Overview

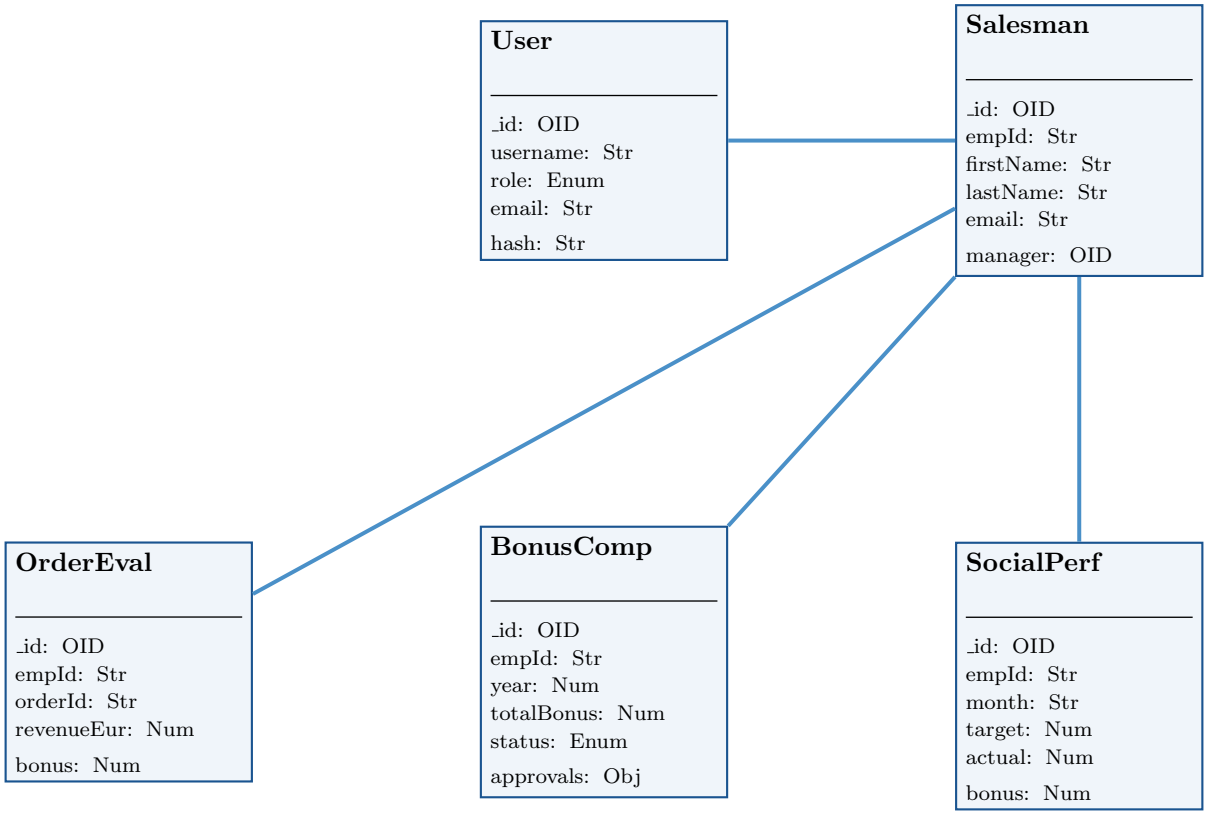


Figure 4.1: Entity Relationship Diagram

4.2 User Model

```
{
  _id: ObjectId,
  username: String (unique),
  email: String,
  passwordHash: String,
  role: Enum ["CEO", "HR", "SALESMAN", "ADMIN"],
  employeeId: String,
  firstName: String,
  lastName: String,
  isActive: Boolean,
  createdAt: Date,
  updatedAt: Date
}
```

Listing 4.1: User Schema Definition

4.3 Bonus Computation Model

```
{
  _id: ObjectId,
  employeeId: String,
  year: Number,
  socialBonus: Number,
  orderBonus: Number,
  totalBonus: Number,
  status: Enum ["PENDING_CEO", "PENDING_HR", "RELEASED"],
  approvedByCEO: { date: Date, remarks: String },
  approvedByHR: { date: Date, remarks: String },
  workflowInstanceId: String (Camunda),
  createdAt: Date,
  updatedAt: Date
}
```

Listing 4.2: BonusComputation Schema

Chapter 5 API Reference

5.1 Authentication

5.1.1 Login Endpoint

POST /api/v1/auth/login

```
# Request
POST /api/v1/auth/login
Content-Type: application/json

{
  "username": "ceo",
  "password": "Ceo123!"
}

# Response (200)
{
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "user": {
    "id": "user_123",
    "username": "ceo",
    "role": "CEO",
    "employeeId": "E001"
  }
}
```

Listing 5.1: Login Request/Response

5.2 Salesmen Management

5.2.1 List Salesmen

GET /api/v1/salesmen

Parameter	Type	Description
Authorization	Header	Bearer token (required)
page	Query	Page number (default: 1)
limit	Query	Results per page (default: 20)

Table 5.1: Salesmen List Parameters

5.2.2 Create Salesman

POST /api/v1/salesmen

```
POST /api/v1/salesmen
Authorization: Bearer <token>
Content-Type: application/json

{
  "firstName": "John",
  "lastName": "Doe",
  "employeeId": "E1001",
  "email": "john@company.com",
  "department": "Sales"
}
```

Listing 5.2: Create Salesman Request

5.3 Bonus Computation

5.3.1 Compute Bonus

POST /api/v1/bonus/:employeeId/compute

This endpoint calculates the bonus based on performance records and orders.

```
POST /api/v1/bonus/E1001/compute
Authorization: Bearer <token>
Content-Type: application/json

{
  "year": 2025,
  "reason": "Q1 Final Calculation"
}

# Response (200)
{
  "employeeId": "E1001",
  "year": 2025,
  "socialBonus": 18000,
  "orderBonus": 5000,
  "totalBonus": 23000,
  "status": "PENDING_CEO"
}
```

Listing 5.3: Bonus Computation Request

5.3.2 Approval Endpoints

Endpoint	Purpose
`POST /api/v1/bonus/:employeeId/approve/ceo`	CEO approval
`POST /api/v1/bonus/:employeeId/approve/hr`	HR approval & storage
`GET /api/v1/bonus/:employeeId`	Get bonus details
`GET /api/v1/bonus/:employeeId/history`	View history

Table 5.2: Bonus API Endpoints

Chapter 6 Workflow Orchestration

6.1 Bonus Approval Workflow

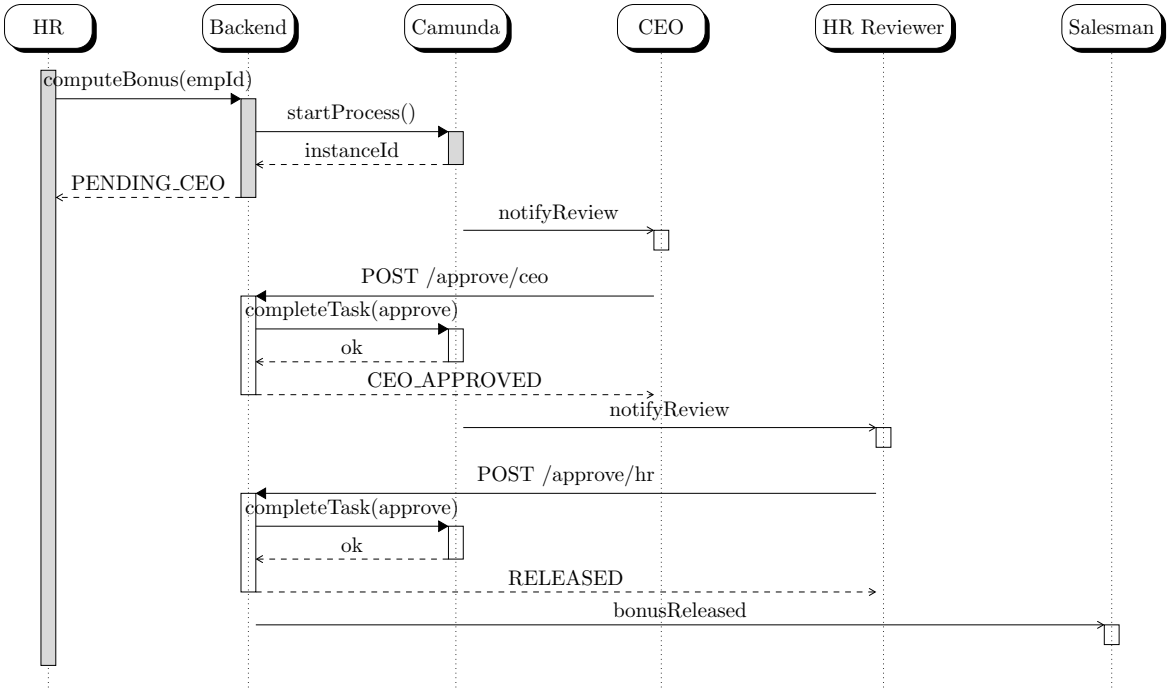


Figure 6.1: Bonus Approval BPMN Workflow

6.2 Camunda Integration

The system uses Camunda BPM for process orchestration. Key features:

- Workflow instantiation via `CamundaService`
- Process variable management
- Task assignment to roles
- Audit trail for all process steps

Note

The BPMN file is located at `workflow/bonusApproval.bpmn` and can be deployed via Camunda Modeler or REST API.

Chapter 7 Setup & Deployment

7.1 Prerequisites

System Requirements

- Node.js 18+ (18.18.0 recommended)
- npm 9+
- Docker & Docker Compose
- Git
- 2GB RAM minimum
- 5GB disk space minimum

7.2 Development Setup

7.2.1 Step 1: Infrastructure

```
# From project root
docker-compose up -d

# Verify services
docker-compose ps
```

Listing 7.1: Start MongoDB and Camunda

7.2.2 Step 2: Backend Setup

```
cd backend
npm install
cp ../.env.example .env
npm run seed            # Seed demo data
npm run dev             # Start dev server
# API available at http://localhost:3000/api/v1
```

Listing 7.2: Backend Installation

7.2.3 Step 3: Frontend Setup

```
cd frontend
npm install
npm start                # Start dev server
# Frontend available at http://localhost:4200
```

Listing 7.3: Frontend Installation

7.3 Production Deployment

7.3.1 Docker Build

```
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci --only=production

COPY src ./src
COPY openapi ./openapi

EXPOSE 3000
CMD ["node", "src/server.js"]
```

Listing 7.4: Backend Dockerfile

7.3.2 Environment Configuration

Variable	Example	Description
NODE_ENV	production	Environment mode
PORT	3000	Server port
MONGODB_URI	mongodb://localhost:27017	Database URL
JWT_SECRET	your-secret-key	JWT signing secret
CORS_ORIGIN	https://app.example.com	CORS allowed origin
CAMUNDA_URL	http://camunda:8080	Camunda engine URL

Table 7.1: Environment Variables

Chapter 8 External Integrations

8.1 OpenCRX Integration

Orders are fetched from OpenCRX and cached for bonus calculation. The integration provides:

- Order data retrieval (JSON-RPC)
- Client ranking information
- Revenue and probability metrics
- Bonus calculation from orders

Chapter 9 Testing & Quality

9.1 Test Architecture

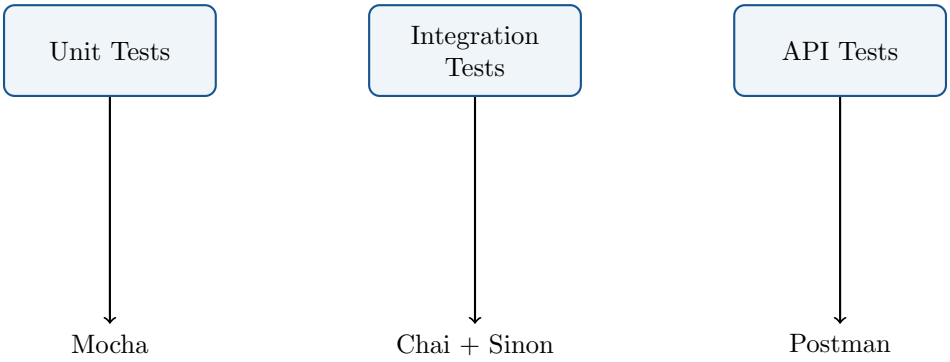


Figure 9.1: Testing Framework Stack

9.2 Running Tests

```
# Unit and integration tests
cd backend
npm test

# Coverage report
npm test -- --coverage

# Watch mode
npm test -- --watch
```

Listing 9.1: Test Execution

9.3 Test Suites

Suite	File	Coverage
Bonus Service	`bonus.service.test.js`	Calculations
Dependencies	`dependencies.test.js`	Integrations
Integration	`integration/`	Full workflows

Table 9.1: Test Suite Organization

Chapter 10 Troubleshooting & Support

10.1 Common Issues

10.1.1 MongoDB Connection Failed

Issue: Cannot Connect to MongoDB

Solution:

1. Check if MongoDB container is running: `docker-compose ps`
2. Restart MongoDB: `docker-compose restart mongodb`
3. Verify URI in .env file
4. Check port 27017 availability

10.1.2 Backend Won't Start

```
# Check port conflicts
netstat -ano | findstr :3000

# Clear dependencies and reinstall
rm -rf node_modules package-lock.json
npm install
npm run dev

# Check logs
npm run dev 2>&1 | head -100
```

Listing 10.1: Debugging Backend Issues

10.1.3 Frontend Build Errors

```
# Clear Angular cache
rm -rf node_modules/.angular

# Rebuild
npm install
npm start
```

Listing 10.2: Fixing Angular Build Issues

10.2 Performance Optimization

- **Database Indexing** — Create indexes on frequently queried fields
- **Response Compression** — Enable gzip in Express
- **Caching** — Implement Redis for session management
- **API Rate Limiting** — Prevent abuse
- **Connection Pooling** — MongoDB connection optimization

10.3 Security Checklist

Production Security Checklist

- ☐ Change JWT_SECRET to strong random value
- ☐ Use HTTPS/TLS in production
- ☐ Secure MongoDB with authentication
- ☐ Implement rate limiting
- ☐ Enable CORS properly
- ☐ Validate all user inputs
- ☐ Use environment variables for secrets
- ☐ Regular dependency updates
- ☐ Enable audit logging
- ☐ Rotate credentials periodically

Chapter 11 Bonus Calculation Algorithm

11.1 Overview

The bonus calculation system computes bonuses based on two components:

1. **Social Performance Bonus** — Based on actual vs. target metrics
2. **Order-Based Bonus** — Based on closed deals and revenue

11.2 Social Performance Calculation

11.2.1 Formula

The social performance bonus is calculated using:

$$\text{Bonus}_{\text{social}} = \sum_{i=1}^{n} \frac{(\text{Actual}_i - \text{Target}_i)}{\text{Target}_i} \times 100,000$$

Where:

- Actual_i = Actual achieved metric for period i
- Target_i = Target metric for period i
- $100,000$ = Base bonus amount

11.2.2 Example

Month	Target	Actual	Bonus Contribution
January	100,000	120,000	20,000 EUR
February	100,000	110,000	10,000 EUR
March	100,000	100,000	0 EUR
Total	300,000	330,000	30,000 EUR

Table 11.1: Social Performance Bonus Example

11.2.3 Implementation

```
async function computeSocialRecordBonus(record) {
  const { target, actual } = record;
  if (!target || target === 0) return 0;

  const variance = (actual - target) / target;
  const bonus = variance * 100000;

  return Math.max(0, Math.round(bonus));
}
```

Listing 11.1: Bonus Calculation Implementation

11.3 Order-Based Bonus Calculation

11.3.1 Formula

Order bonuses consider revenue and closing probability:

$$\text{Bonus}_{\text{order}} = \text{Revenue} \times \text{Probability} \times 0.05$$

Where:

- Revenue = Order value in EUR
- Probability = Closing probability (0-100%)
- 0.05 = Commission rate

11.3.2 Example

Order	Revenue	Probability	Bonus
ORD001	50,000 EUR	95%	2,375 EUR
ORD002	30,000 EUR	80%	1,200 EUR
ORD003	20,000 EUR	60%	600 EUR
Total	100,000 EUR	—	4,175 EUR

Table 11.2: Order Bonus Calculation Example

11.4 Total Bonus Computation

$$\text{Total Bonus} = \text{Bonus}_{\text{social}} + \text{Bonus}_{\text{order}}$$

The system persists this as a `BonusComputation` record with status tracking through the approval workflow.

11.5 Bonus Constraints

- Minimum bonus: 0 EUR (no negative bonuses)
- Maximum bonus: 500,000 EUR (configurable)
- Precision: 2 decimal places
- Year-based: Annual calculations with monthly/quarterly breakdown

Chapter 12 Workflow & Approval Process

12.1 BPMN Workflow Overview

The bonus approval workflow is implemented using Camunda BPM with the following states:

12.1.1 Workflow State Machine

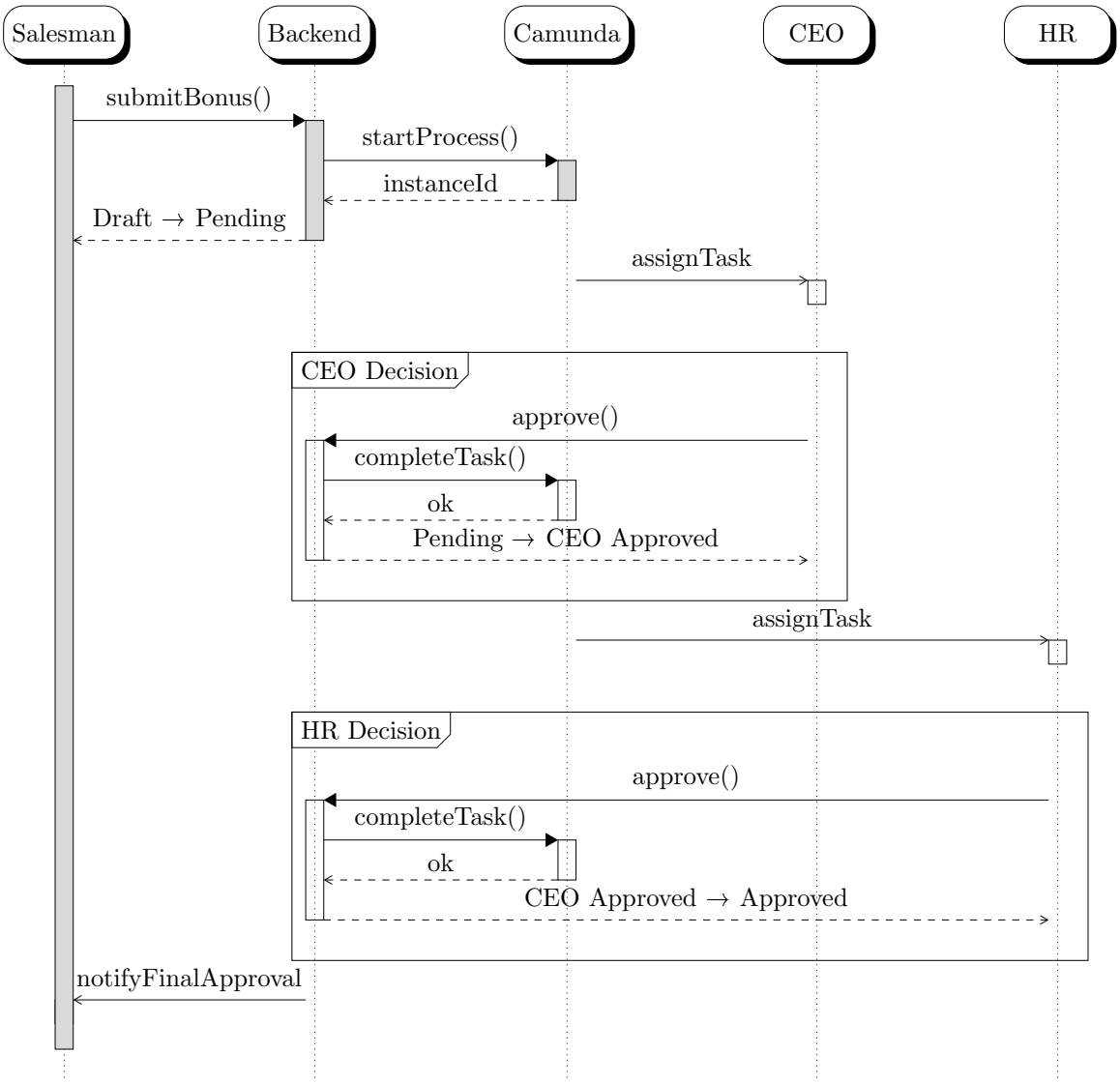


Figure 12.1: Bonus Approval Workflow

12.2 Workflow States

[labelindent=1cm, font=]- Initial state when bonus computation is created - Submitted for CEO review - CEO has reviewed and approved - Final approval from HR (Approved state) - Rejected by CEO or HR, returned to Draft

12.3 Approval Task Details

12.3.1 CEO Review Task

CEO Review Process

Draft **Pending CEO** **Approved** **HR Approved** **Rejected**

- **Owner:** CEO (role-based assignment)
- **Deadline:** 5 business days
- **Actions:**
  - Review bonus computation summary
  - Check total bonus amount against budget
  - Approve or reject with comments
- **Exit Criteria:**
  - Approved → forward to HR
  - Rejected → return to draft with feedback

12.3.2 HR Review Task

HR Final Approval Process

- **Owner:** HR Team (role-based assignment)
- **Deadline:** 3 business days
- **Actions:**
  - Verify all data completeness
  - Check compliance with company policies
  - Final approval or rejection
- **Exit Criteria:**
  - Approved → bonus marked as finalized
  - Rejected → return to draft

12.4 Workflow API Endpoints

Method	Endpoint	Action	Description
POST	`/api/workflow/{id}/approve`	CEO/HR Approve	Approve bonus computation
POST	`/api/workflow/{id}/reject`	Reject	Return to draft with reason
GET	`/api/workflow/{id}/history`	Get History	View state transitions
GET	`/api/workflow/tasks`	Get Tasks	List pending tasks

Table 12.1: Workflow API Endpoints

12.5 Integration with Services

12.5.1 Camunda Service Integration

The workflow is managed through the `CamundaService`:

```
// Start workflow
await camundaService.startProcess('bonus-approval', {
  bonusId: computation.id,
  totalAmount: computation.total,
  salesman: computation.salesman
});

// Complete CEO approval task
await camundaService.completeTask(taskId, {
  action: 'approve',
  comments: 'Approved by CEO',
  userId: req.user.id
});
```

Listing 12.1: Camunda Service Integration Example

12.6 Notifications & Escalations

12.6.1 Email Notifications

- **To CEO:** When bonus reaches pending state
- **To HR:** When CEO approves
- **To Salesman:** Final approval notification
- **Escalation:** If approval exceeds deadline

12.6.2 Escalation Rules

Escalation Triggers

- CEO review exceeds 5 days → escalate to CFO
- HR review exceeds 3 days → escalate to Head of HR
- Rejected twice → manual review required

12.7 Workflow Monitoring

Access workflow metrics at: `http://localhost:8080/camunda`

- Active process instances
- Running tasks
- Completed milestones
- Performance metrics

Chapter A Quick Reference

A.1 API Endpoints Summary

Method	Endpoint	Purpose
POST	/api/v1/auth/login	User login
GET	/api/v1/salesmen	List salesmen
POST	/api/v1/salesmen	Create salesman
GET	/api/v1/bonus/:id	Get bonus
POST	/api/v1/bonus/:id/compute	Compute bonus
POST	/api/v1/bonus/:id/approve/ceo	CEO approval
POST	/api/v1/bonus/:id/approve/hr	HR approval
GET	/api/v1/performance/social/:id	Get performance
POST	/api/v1/performance/social	Create record
GET	/api/v1/orders/:id	Get orders
GET	/api/v1/health-check	System health

Table A.1: API Endpoints Reference

A.2 Demo Credentials

Role	Username	Password
CEO	ceo	Ceo123!
HR	hr	Hr123!
Salesman	salesman01	password123

Table A.2: Demo User Credentials

A.3 Useful Commands

```
# Start infrastructure
docker-compose up -d

# Backend operations
cd backend && npm install && npm run seed && npm run dev

# Frontend operations
cd frontend && npm install && npm start

# Run tests
npm run backend:test

# Check health
curl http://localhost:3000/api/v1/health-check

# View API docs
# Open: http://localhost:3000/api-docs
```

Listing A.1: Quick Reference Commands

Chapter B Glossary

BPMN Business Process Model and Notation — Standard for workflow design

RBAC Role-Based Access Control — Security model for authorization

JWT JSON Web Token — Stateless authentication method

REST Representational State Transfer — API architectural style

ORM Object-Relational Mapping — Mongoose provides this for MongoDB

JSON-RPC JSON Remote Procedure Call — Lightweight RPC protocol

OpenAPI API specification standard for REST services

CORS Cross-Origin Resource Sharing — HTTP-based access control